

UNIVERSITY PARTNER



Flower Classification: CNN Architecture Comparison

6CS012 Artificial Intelligence and Machine Learning

University ID	: 2331250
Student Name	: Riddhi Maskey
Group	: L6CG13
Tutor	: Durga Pokharel
Module Leader	: Simon Giri
Submitted on	: <05/13/2025>
Total no. of words	: 2268

Abstract

This report analyzes how the different architectures work for image classification using the flower dataset. The Convolutional Neural Network (CNN) is built using two architectures: first, a baseline model, and second, a deeper model with regularization techniques such as dropout and batch normalization. The model is further developed using a pre-trained model from the ImageNet to fine-tune the model for transfer learning using two methods: first, feature extraction, and then a fine-tuning step. The results indicated that both the deeper CNN and the baseline model were trained and tested. The baseline model achieved an accuracy of 73%, while the deeper model reached 74% accuracy. However, the VGG16 model using transfer learning performed even better, achieving an accuracy of 88.08% after fine-tuning.

Table of Contents

1	Introduction	1
2	Dataset.....	1
2.1	Source:	1
2.2	Structure of the Dataset:	1
2.3	Preprocessing Processes	2
3	Methodology	3
3.1	Baseline Model:	3
3.2	Deeper Model:	5
4	Experiments and Results	5
4.1	Baseline vs. Deeper Architecture	6
4.2	Computational Efficiency.....	7
4.3	Training with Different Optimizers	8
4.4	Challenges in Training	9
5	Fine-Tuning or Transfer Learning	9
6	Conclusion	10

Table of Figures

Figure 1: Total number of image after data cleaning	2
Figure 2: Image distribution per set	2
Figure 3: Data augmented images	3
Figure 4: Accuracy of baseline model.....	4
Figure 5: Loss of baseline model	4
Figure 6: Accuracy and loss of the deeper model	5
Figure 7: Accuracy and loss of baseline model	6
Figure 8: Accuracy and loss of deeper model	6
Figure 9: Classification report of both baseline and deeper model.....	7
Figure 10: Model's Performance	7
Figure 11: Computational performance	8
Figure 12: Accuracy and loss of deeper model with SGD and Adam optimizer.....	8
Figure 13: Model's Comparison summary.....	10

1 Introduction

Image classification represents a task in computer vision that focuses on identifying and organizing the visual data automatically by categorizing images into specific classes or labels according to their visual features of the images, and is a problem in the context of its application in autonomous vehicles, medical diagnosis, and content moderation. The main focus of this project is to classify the flower images. The dataset that is used in this project is a multi-class classification flower dataset, consisting of a total of 4312 images of different flower species: daisy, dandelion, rose, sunflower, and tulip, where each class contains around 700 or more images.

Deep learning is a type of machine learning that uses layers of features to classify data. It's particularly effective for tasks like image classification, object recognition, and segmentation. In these tasks, recognizing patterns is crucial, and this involves extracting features to assess how well the model performs. Convolutional Neural Networks (CNNs) are important in this area because they learn features directly from image pixels, making them great for visual data analysis.

The goal of this project is to compare a baseline model, a deeper model, and the fine-tuning method on the dataset and evaluate their performance in terms of accuracy, loss, computational efficiency, convergence speed, and training challenges.

2 Dataset

2.1 Source:

The dataset used for this project is the Flower Classification dataset, which was selected from the recommended dataset for the image classification tasks.

2.2 Structure of the Dataset:

The dataset consists of five classes and contains a total of 4312 images, with a minimum of 730 images per class. While checking for corrupted images, a total of 35 images, 7 per class, were found and removed from the dataset. Now the total number of images in the dataset after data cleaning is 4277.

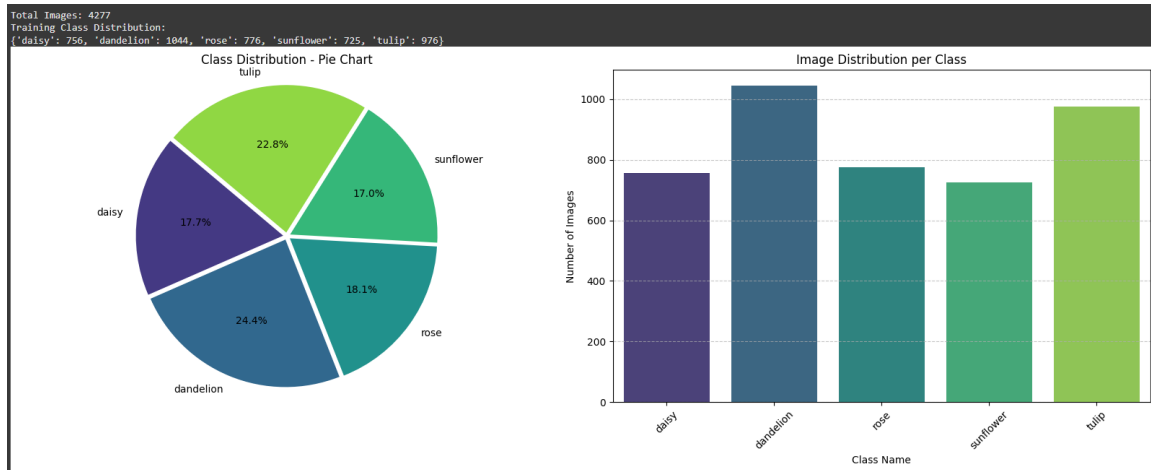


Figure 1: Total number of image after data cleaning

2.3 Preprocessing Processes

After cleaning the dataset, the data was further divided into the training set, Validation set, and test set in a 70:15:15 ratio and saved them in the newly created folder with respective names. There is a total of 2992 images in the train set, 639 in the validation set, and 646 images in the test set.

```
Train Set Class Distribution:
{'dandelion': 730, 'daisy': 529, 'rose': 543, 'sunflower': 507, 'tulip': 683}
Val Set Class Distribution:
{'dandelion': 156, 'daisy': 113, 'rose': 116, 'sunflower': 108, 'tulip': 146}
Test Set Class Distribution:
{'dandelion': 158, 'daisy': 114, 'rose': 117, 'sunflower': 110, 'tulip': 147}
```

Figure 2: Image distribution per set

Preprocessing techniques were applied, which include normalization, meaning scaling pixel values of images to a range of 0 – 1, and data augmentation methods to the training set. The resolution of all the images was resized to 224*224 pixels during preprocessing. This ensures a clean and diversified dataset. Data augmentation was done on the training dataset, using these steps: rotation, width shift, height shift, shear range, zoom range, horizontal flip, and vertical flip to improve generalization.



Figure 3: Data augmented images

3 Methodology

3.1 Baseline Model:

The baseline model was built with one input layer, three blocks of convolutional layers, which are followed by one max pooling layer for each convolutional layer with 32, 64, 128 filters of size 3*3 in increasing order respectively, which helps to select the maximum value within each pooling window with 2*2 filter, reducing the spatial dimensions by half and extracting the dominant features. After this, there is one flatten layer which is used to convert the 3D feature maps to a 1D vector for the fully connected dense layer, three dense layers with 128, 64, 32 filters in decreasing order respectively, and one dense output layer at the end with number of class neurons (5 in this dataset). The dense layers learn extracted high-level features from the previous convolutional layers.

The ReLu (Rectified Linear Unit) activation function was used in the convolutional and dense layers as it helps non-linearity and in learning complex patterns. The softmax activation function is used on the final dense layer, as it converts the output into probabilities for each flower class and uses the sparse categorical cross-entropy for the loss function, as the labels are integers. In this architecture, various hyperparameters were used. The input size of images is 224*224 pixels with 3 color channels(RGB), batch

size of 32, and 30 epochs for training the model. Adam is selected as the optimizer to compile this model, as it is efficient and has an adaptable learning rate with a default value of 0.001. The EarlyStopping and ModelCheckpoint callbacks are used to stop training the model if the validation does not decrease over the five epochs, as patience is set to five, and save the best model during the training process.

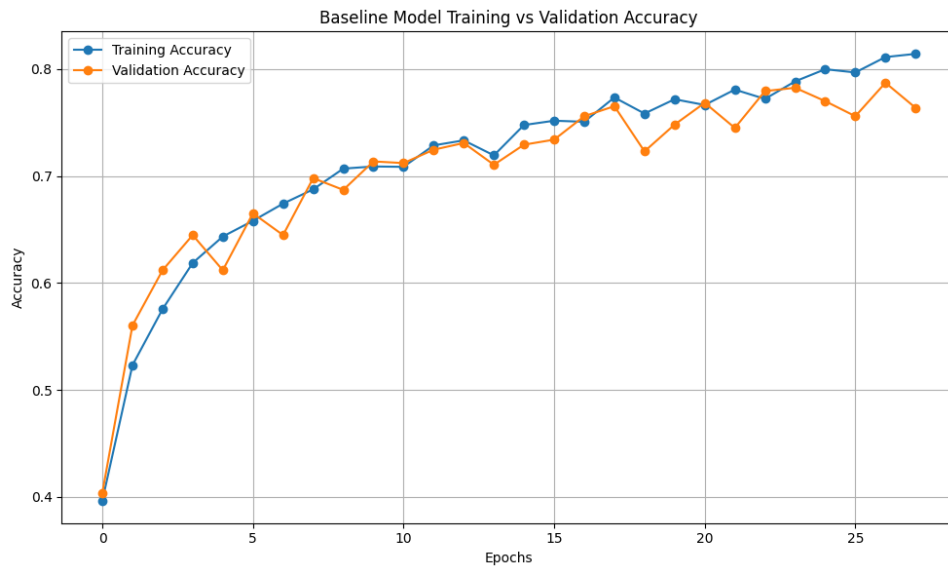


Figure 4: Accuracy of baseline model

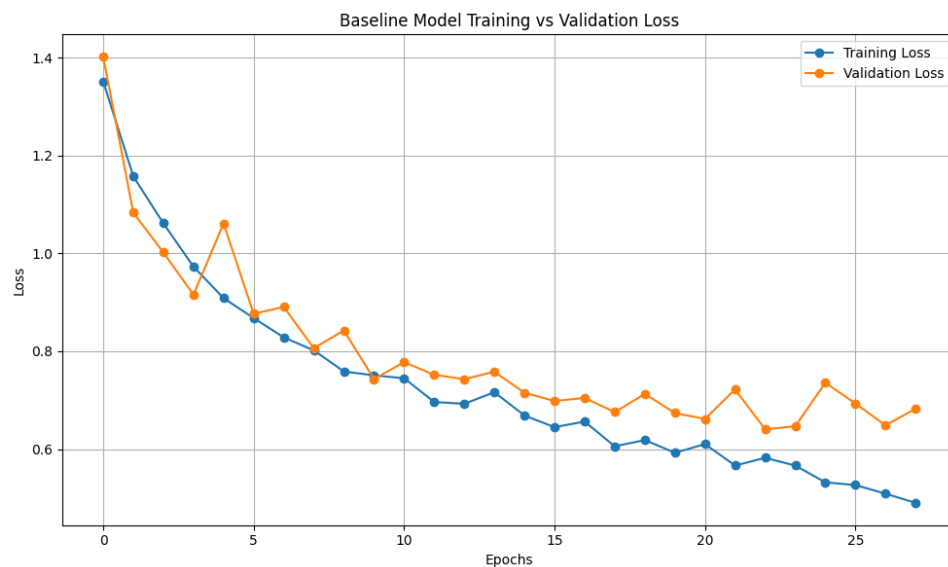


Figure 5: Loss of baseline model

3.2 Deeper Model:

After building the baseline model from scratch, the deeper model was built by adding layers and regularization techniques to the baseline model, making it a complex network. The Deeper model was built with one input layer, three convolutional blocks, each with two convolutional layers, and batch normalization for speeding up training and improving convergence, which are followed by one max pooling layer and dropout layers. All three blocks use the same padding, ReLu activation, 3*3 kernels with 32, 64, 128 filters per block in the respective order. The rate of the dropout layers is increasing as it goes to the next block, 0.25, 0.3, 0.4, respectively, to prevent overfitting, as it randomly sets a fraction of input units to zero during training. And L2 regularization is set to the first convolutional layer.

A Global Average Pooling (GAP) was used to reduce the number of parameters and computational complexity in comparison to the flatten layer. Two dense layers are set up after the GAP layer, which is followed by one dropout layer(0.5) after the first dense layer, 128, 64, respectively, with ReLU activation function and a final output dense layer with a softmax activation function. In this architecture, the same hyperparameters and optimizer that were used in the baseline model are applied here as well.

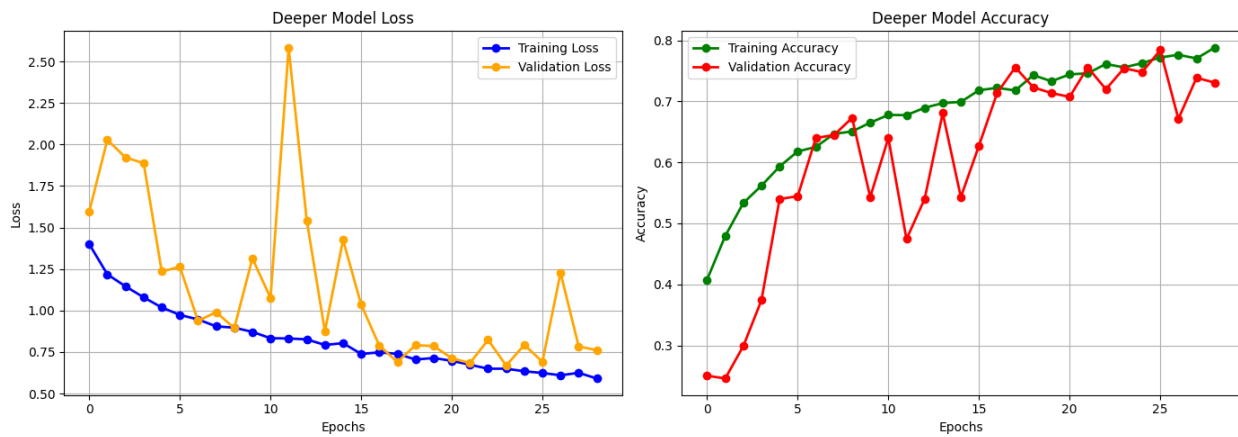


Figure 6: Accuracy and loss of the deeper model

4 Experiments and Results

The different experiment was performed between the models: Baseline model and Deeper model.

4.1 Baseline vs. Deeper Architecture

The baseline model was set up to run for 30 epochs, since the validation loss didn't improve after epoch 23, early stopping occurred at 28 epochs. The final test accuracy of this model is 73.2%, and the test loss of 76%. The deeper model was set up to run for 30 epochs, since the validation loss didn't improve after epoch 24, early stopping occurred at 29 epochs. The final test accuracy of this model is 73.8%, and the test loss of 71%. It shows that the deeper model performs better than the baseline model in terms of test accuracy, with a little 0.6% improvement.

```
-----  
Test Loss: 0.7608528137207031  
Test Accuracy: 0.7321981191635132  
-----  
Train Loss: 0.5314769744873047  
Train Accuracy: 0.7957887649536133  
-----  
Validation Loss: 0.6402819752693176  
Validation Accuracy: 0.7793427109718323  
-----
```

Figure 7: Accuracy and loss of baseline model

```
-----  
Test Loss: 0.71965491771698  
Test Accuracy: 0.7383900880813599  
-----  
Train Loss: 0.6319171786308289  
Train Accuracy: 0.7613636255264282  
-----  
Validation Loss: 0.6681316494941711  
Validation Accuracy: 0.7543035745620728  
-----
```

Figure 8: Accuracy and loss of deeper model

In the context of evaluation metrics, the model predicted the dandelion class correctly by 70%, as its precision is 0.79, whereas when predicting rose, it makes mistakes as it has a low precision of 0.64. The model predicted 85% of all the sunflower images correctly, while leaving the 15%. In case of a deeper model, the daisies images were predicted and found the real daisies as it indicates it has highest recall and F1-score. And it also predicted the tulips class accurately as it has highest precision

🚩 Baseline Model Classification Report:				
	precision	recall	f1-score	support
daisy	0.72	0.81	0.76	114
dandelion	0.79	0.68	0.73	158
rose	0.61	0.65	0.63	117
sunflower	0.78	0.85	0.82	110
tulip	0.75	0.71	0.73	147
accuracy			0.73	646
macro avg	0.73	0.74	0.73	646
weighted avg	0.74	0.73	0.73	646
21/21 ————— 3s 144ms/step				
🚩 Deeper Model Classification Report:				
	precision	recall	f1-score	support
daisy	0.74	0.82	0.78	114
dandelion	0.83	0.72	0.77	158
rose	0.56	0.83	0.67	117
sunflower	0.76	0.81	0.78	110
tulip	0.89	0.57	0.70	147
accuracy			0.74	646
macro avg	0.76	0.75	0.74	646
weighted avg	0.77	0.74	0.74	646

Figure 9: Classification report of both baseline and deeper model

The baseline model took 26.29 minutes to train the model whereas the deeper model took 28.08 minutes with the additional layer and regularization techniques. As a conclusion, the deeper model performed a bit better than the baseline model.

Metric	Baseline Model	Deeper Model	Difference
Test Accuracy	73.2%	73.8%	+0.6%
Training Accuracy	79.6%	76.1%	-3.5%
Validation Accuracy	77.9%	75.4%	-2.5%
Test Loss	0.761	0.720	-5.4%
Training Time	26.29 min	28.08 min	+6.8%

Figure 10: Model's Performance

4.2 Computational Efficiency

Google Colab (GPU acceleration) was used as hardware while training the model.

Resource	Baseline	Deeper Model
Training Time	26.29 min	28.08 min
GPU Memory	7.8GB	8.4GB
CPU Utilization	62%	68%

Figure 11: Computational performance

- Training Time:
 - Baseline Model: It takes around 26.29 minutes to train this model.
 - Deeper Model: It takes around 28.08 minutes to train this model.
- Memory Usage: The baseline model uses around 4.1 GB of memory, whereas the deeper model uses around 8.1 GB of memory, which is more in comparison to the baseline model as it has more additional layers and parameters.
- The callbacks, like earlystopping, is used to minimize the training time and to prevent the overfitting of data.

4.3 Training with Different Optimizers

To know how the different optimizers affect the model performance, the deeper model was trained with two optimizers, one Adam and the other one is SGD (Stochastic Gradient Descent) optimizer.

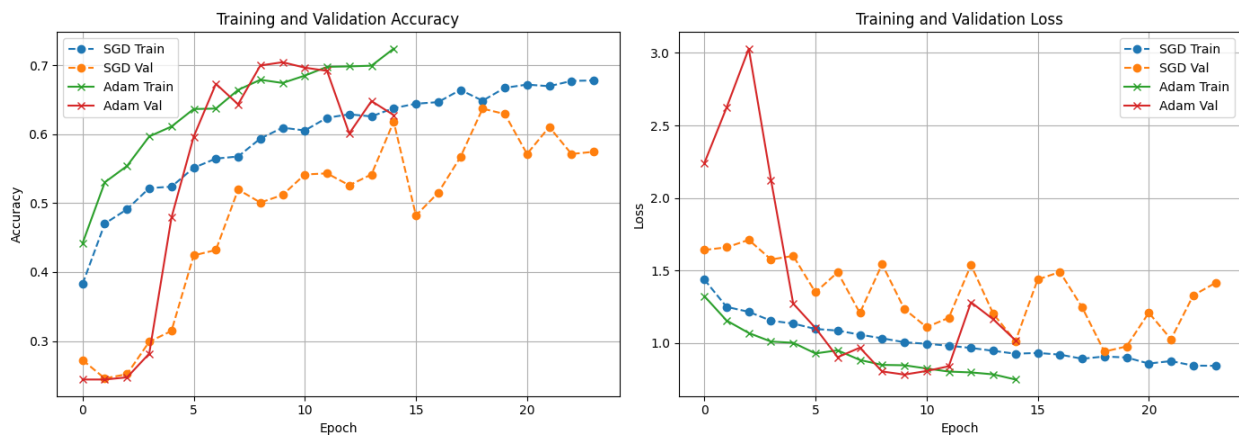


Figure 12: Accuracy and loss of deeper model with SGD and Adam optimizer

As you can see from the image, Adam optimizer trains the model faster and more stable with better accuracy and lower loss in the final performance for both training and validation sets. SGD optimizer shows more instability in validation performance, with low accuracy and slow learning. There is a smaller gap between the training and validation metrics for Adam, pointing to better generalization.

4.4 Challenges in Training

During the training of the baseline model, the validation loss stopped improving after the 18th epoch, indicating overfitting, where the model performed well on training data but poorly on test data. This suggested the dataset was too simple or limited. The deeper model showed the same issue, so batch normalization and dropout techniques were applied. Initially, the baseline model also suffered from underfitting, failing to capture features effectively. To address this, more layers and filters were added, which improved feature extraction. The training took 28.08 minutes, and memory usage was high.

5 Fine-Tuning or Transfer Learning

We used the VGG16 model, which is already trained on a large dataset called ImageNet, for transfer learning. This process had two parts: first, we extracted features, and then we fine-tuned the model. We chose VGG16 because it has a strong track record for classifying images and works well with different types of data..

The first step involved freezing VGG16 base layers and fitting only the classification layers on top. The model was trained within 15 epochs with an Adam optimizer, which is set at a learning rate of 0.0001, and EarlyStopping was used which resulting in a test accuracy of 86.07%. During the fine-tuning phase, we adjusted the earlier convolutional block by unfreezing it and training it with a lower learning rate. This helped it adapt better to our specific data while minimizing the risk of forgetting what it learned and overfitting. As a result, our test accuracy increased to 88.08%, showing that transfer learning enhanced our model's performance.

```
Model Comparison Summary:
-----
Fine-tuned VGG16 Model Accuracy: 0.8808
Feature Extraction VGG16 Model Accuracy: 0.8607
Part A Baseline Model Accuracy: 0.7321
Part A Deeper Model Accuracy: 0.7383

Training Time Comparison:
Fine-tuning Training Time: 618.40 seconds
Feature Extraction Training Time: 942.32 seconds
Part A Baseline Training Time: 26.29 minutes
Part A Deeper Model Training Time: 28.08 minutes
```

Figure 13: Model's Comparison summary

6 Conclusion

This project was built from scratch, known as a baseline model. Additionally, build a deeper model by extending the baseline model by adding an extra layer and regularization techniques like batch normalization, dropout. Then the model was fine-tuned with transfer learning by using pre-trained VGG16 model from ImageNet.

The basic CNN set a good starting point with a validation accuracy of 73%. The more advanced CNN performs a bit better at 74% validation accuracy by adding more layers, dropout, and L2 regularization. This deeper model showed improved feature extraction and accuracy for certain classes (like tulips), but it had a training time limit of 28.08 minutes and a high computational cost, highlighting the balance needed between complexity and efficiency. The VGG16 model provided a much bigger boost, reaching 86.07% accuracy and a validation accuracy of 88.08% after fine-tuning. This shows the benefits of using pre-trained models, especially with smaller datasets or fewer resources. Despite the good results, the project faced several problems. Overfitting was a major issue, particularly with the deeper architecture, where the validation loss stabilized very quickly. Even after adding batch normalization and dropout, we might need more adjustments or regularization. There was also a problem with class imbalance, which affected the model's fairness, especially for the rose class, which had the lowest precision.